

claude-windows / e4ec78ed-5ae4-4c65-b01a-7cb8683e0820

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\c--Users-avidu-Projects-sports-data-collector\e4ec78ed-5ae4-4c65-b01a-7cb8683e0820.jsonl`
- SHA-256: `3a0b0d752ea1bcbf80c4b0e536c0be12957f5faabe00a3535a40232eabe8f8ce`
- Source modified: `2026-06-10T10:34:11+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `c--Users-avidu-Projects-sports-data-collector`
- session_id: `e4ec78ed-5ae4-4c65-b01a-7cb8683e0820`
```

Transcript

1. user (2026-06-10T09:58:59.865Z)

In C:\Users\avidu\Projects\sports-data-collector, the CVAT parser src/parsers/cvat/rally_cvat.py (used by `python -m src.cli.curate convert-cvat`) silently drops the `shot\_count` attribute. Reproduction: the Roora CVAT XMLs at %TEMP%\roora\pb\annotations.xml and %TEMP%\roora\tr\annotations.xml have `

2. assistant (2026-06-10T09:59:06.840Z)

I'll fix the CVAT parser's dropped `shot\_count` attribute. Let me start by reading the parser and checking the repro files.

3. user (2026-06-10T09:59:07.805Z)

```
1      """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3      Why this exists
4      -----
5      Our annotation brief (`docs/annotation/ROORA_BRIEF.md`) asks vendors for a
6      per-rally CSV that imports directly via `curate import-local`. A vendor may
7      instead do the work in CVAT and hand back the CVAT export: per-frame bounding
8      boxes, with the rally fields (`rally_number`, `shots_count`,
9      `ending_reason` ...) carried as CVAT object attributes on each box. The box
10     geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11     metadata to.
12
13     This module is the ingestion-side adapter that absorbs that vendor format so the
14     modeling repo never has to. It does two things, both deliberately on our side
15     so we fix mechanical problems rather than renegotiating them:
16
17     1. Recompute time from the frame range. A rally's interval is taken from the
18     first/last frame a box for that rally appears on:
19     `seconds = frame / fps` at the video's true fps. We ignore any
20     `start_time`/`end_time` attributes the vendor wrote ? a CVAT job that
21     samples every step frames and assumes 30 fps yields systematically wrong
22     times (we have seen `start_time == frame / (step * 30)`, i.e. off by the
23     sampling step at the wrong fps). The frame index is reliable; the vendor's
24     seconds are not.
25
```

```

26     2. **Surface ? never silently fix ? content defects.** Overlapping rallies,
27     zero-duration rallies, ending_reason values outside the agreed set, and
28     too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
29     rally boundary would poison ground truth, so that stays a human/vendor call.
30
31     Supported input: the single-file CVAT XML export, in either **image mode**
32     (<image><box>...</box></image>) or **track/interpolation mode** (<track><box frame=...></box></track>).
33     The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
34     later if a vendor sends only those.
35     """
36     from __future__ import annotations
37
38     import os
39     import zipfile
40     import xml.etree.ElementTree as ET
41     from collections import Counter
42     from dataclasses import dataclass, field
43     from typing import Any, Dict, List, Optional, Tuple
44
45     from src.core.parsing import safe_int
46     from src.core.downloader import probe_resolution_fps
47
48     # The agreed cross-sport ending_reason vocabulary for net-separated sports.
49     # Kept here as a soft check only (a warning, never a hard failure) so this
50     # adapter does not depend on the annotation-spec wording.
51     DEFAULT_ALLOWED_REASONS = {
52         "winner",
53         "forced_error",
54         "unforced_error",
55         "service_fault",
56         "let",
57         "other",
58     }
59
60     # The observational outcome of the last shot ? orthogonal to ending_reason
61     # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
62     DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
63
64     # CVAT attribute names we map onto canonical columns. The vendor's CVAT label
65     # schema has used suffixed names like "score_before (A-B)"; we match leniently.
66     _ATTR_ALIASES = {
67         "rally_number": ("rally_number",),
68         "shots_count": ("shots_count",),
69         "ending_reason": ("ending_reason",),
70         "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
71 (in/net/out)"),
72         "score_before": ("score_before", "score_before (a-b)"),
73         "score_after": ("score_after", "score_after (a-b)"),
74         "notes": ("notes", "note"),
75     }
76
77     CANONICAL_COLUMNS = [
78         "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
79         "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
80         "notes",
81     ]
82
83     @dataclass
84     class RallyRecord:
85         """One rally recovered from the CVAT export, time re-derived from frames."""
86         rally_number: int
87         start_time: float          # seconds, = frame_min / fps
88         end_time: float            # seconds, = frame_max / fps (see padding note)
89         shots_count: Optional[int]

```

```

89     ending_reason: Optional[str]
90     last_shot_outcome: Optional[str]
91     score_before: Optional[str]
92     score_after: Optional[str]
93     notes: Optional[str]
94     frame_min: int
95     frame_max: int
96     n_boxes: int
97     vendor_start_time: Optional[str] = None    # the (discarded) attribute value
98     vendor_end_time: Optional[str] = None
99
100    @property
101    def duration(self) -> float:
102        return self.end_time - self.start_time
103
104
105    @dataclass
106    class Issue:
107        """A defect found in the delivery. ``blocker`` issues stop a clean import."""
108        severity: str    # 'blocker' | 'high' | 'medium' | 'low'
109        code: str
110        message: str
111        rally_number: Optional[int] = None
112
113
114    @dataclass
115    class ConversionResult:
116        records: List[RallyRecord]
117        issues: List[Issue]
118        fps: float
119        step: int
120        meta: Dict[str, Any] = field(default_factory=dict)
121
122    @property
123    def n_blockers(self) -> int:
124        return sum(1 for i in self.issues if i.severity == "blocker")
125
126
127    def _mmss(seconds: float) -> str:
128        """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
129        m = int(seconds // 60)
130        s = seconds - m * 60
131        return f"{m}:{s:06.3f}"
132
133
134    def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
135        for alias in _ATTR_ALIASES[canonical]:
136            if alias in attrs and attrs[alias] != "":
137                return attrs[alias]
138        return None
139
140
141    def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
142        """Parse a CVAT XML export into (meta, boxes).
143
144        ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
145        entry per visible box, from either image-mode or track-mode exports.
146        ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
147
148        """
149        if not os.path.exists(xml_path):
150            raise FileNotFoundError(f"CVAT export not found: {xml_path}")
151
152        # Accept the standard CVAT delivery artifact (a .zip containing annotations.xml) as
153        # well as a bare .xml. Corrupt/unparseable XML is surfaced as a ValueError (a clean
154        # gate failure) rather than an uncaught ParseError traceback.

```

```

154     try:
155         if zipfile.is_zipfile(xml_path):
156             with zipfile.ZipFile(xml_path) as zf:
157                 names = ([n for n in zf.namelist() if n.endswith("annotations.xml")]
158                        or [n for n in zf.namelist() if n.endswith(".xml")])
159                 if not names:
160                     raise ValueError(f"CVAT zip contains no annotations.xml:
{xml_path}")
161                 root = ET.fromstring(zf.read(names[0]))
162             else:
163                 root = ET.parse(xml_path).getroot()
164         except ET.ParseError as e:
165             raise ValueError(f"CVAT XML is corrupt/unparseable ({xml_path}): {e}")
166         meta: Dict[str, Any] = {}
167
168         job = root.find("./job")
169         if job is not None:
170             stop = job.findtext("stop_frame")
171             if stop is not None and stop.strip().isdigit():
172                 meta["stop_frame"] = int(stop.strip())
173             ff = job.findtext("frame_filter") or ""
174             if "step=" in ff:
175                 try:
176                     meta["step"] = int(ff.split("step=")[1].split()[0])
177                 except (ValueError, IndexError):
178                     pass
179
180         def _attrs(el: ET.Element) -> Dict[str, str]:
181             out: Dict[str, str] = {}
182             for a in el.findall("attribute"):
183                 out[(a.get("name") or "").strip().lower()] = (a.text or "").strip()
184             return out
185
186         boxes: List[Dict[str, Any]] = []
187
188         # Image mode: <image id=frame> ... <box>...</box>
189         for img in root.findall("image"):
190             try:
191                 frame = int(img.get("id"))
192             except (TypeError, ValueError):
193                 continue
194             for box in img.findall("box"):
195                 boxes.append({"frame": frame, "attrs": _attrs(box)})
196
197         # Track mode: <track> ... <box frame=N outside=0>...</box>
198         for track in root.findall("track"):
199             for box in track.findall("box"):
200                 if box.get("outside") == "1":
201                     continue # a track's terminating keyframe carries no real label
202                 try:
203                     frame = int(box.get("frame"))
204                 except (TypeError, ValueError):
205                     continue
206                 boxes.append({"frame": frame, "attrs": _attrs(box)})
207
208         return meta, boxes
209
210
211     def _infer_step(frames: List[int]) -> int:
212         """Most common positive gap between consecutive distinct sampled frames."""
213         uniq = sorted(set(frames))
214         gaps = [b - a for a, b in zip(uniq, uniq[1:]) if b - a > 0]
215         if not gaps:
216             return 1
217         return Counter(gaps).most_common(1)[0][0]

```

```

218
219
220 def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
221     """Group boxes by rally_number and derive each rally's frame range -> seconds."""
222     if fps is None or fps <= 0:
223         raise ValueError("fps must be a positive number to convert frames to seconds")
224
225     by_rally: Dict[int, List[Dict[str, Any]]] = {}
226     for b in boxes:
227         rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
228         if rn is None:
229             continue
230         by_rally.setdefault(rn, []).append(b)
231
232     records: List[RallyRecord] = []
233     for rn in sorted(by_rally):
234         items = by_rally[rn]
235         frames = sorted(b["frame"] for b in items)
236         attrs = items[0]["attrs"]
237         fmin, fmax = frames[0], frames[-1]
238         records.append(RallyRecord(
239             rally_number=rn,
240             start_time=fmin / fps,
241             end_time=fmax / fps,
242             shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
243             ending_reason=_attr_lookup(attrs, "ending_reason"),
244             last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
245             score_before=_attr_lookup(attrs, "score_before"),
246             score_after=_attr_lookup(attrs, "score_after"),
247             notes=_attr_lookup(attrs, "notes"),
248             frame_min=fmin,
249             frame_max=fmax,
250             n_boxes=len(frames),
251             vendor_start_time=attrs.get("start_time"),
252             vendor_end_time=attrs.get("end_time"),
253         ))
254     return records
255
256
257 def detect_issues(records: List[RallyRecord],
258                 stop_frame: Optional[int] = None,
259                 allowed_reasons: Optional[set] = None,
260                 allowed_outcomes: Optional[set] = None,
261                 max_gap_s: float = 15.0) -> List[Issue]:
262     """Find the defects that block or degrade a clean ground-truth import.
263
264     Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
265     slack of ~0.5s is acceptable for this pipeline because the downstream
266     highlight compiler pads every clip on output (badminton-highlight-indexer,
267     backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
268     absorbs the sampling uncertainty when highlights are trimmed.
269     """
270     allowed = allowed_reasons if allowed_reasons is not None else
DEFAULT_ALLOWED_REASONS
271     allowed_out = allowed_outcomes if allowed_outcomes is not None else
DEFAULT_ALLOWED_OUTCOMES
272     issues: List[Issue] = []
273
274     ordered = sorted(records, key=lambda r: r.rally_number)
275     for i, r in enumerate(ordered):
276         if r.end_time <= r.start_time:
277             issues.append(Issue(
278                 "blocker", "zero_duration",
279                 f"Rally {r.rally_number}: end <= start "
280                 f"({frames {r.frame_min}-{r.frame_max}}); single-sample or stray box.",

```

```

281         r.rally_number))
282
283     if r.ending_reason is not None and r.ending_reason not in allowed:
284         issues.append(Issue(
285             "medium", "ending_reason_vocab",
286             f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
287             f"not in the agreed set {sorted(allowed)}.",
288             r.rally_number))
289
290     if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
291         issues.append(Issue(
292             "low", "outcome_vocab",
293             f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
294             f"not in {sorted(allowed_out)}.",
295             r.rally_number))
296
297     # Inflated span: a long rally with implausibly few shots usually means a
298     # stray/duplicated box stretched the frame range past the real ending.
299     if r.shots_count and r.duration >= 5.0:
300         sps = r.shots_count / r.duration
301         if sps < 0.3:
302             issues.append(Issue(
303                 "medium", "shot_density",
304                 f"Rally {r.rally_number}: {r.shots_count} shots over "
305                 f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
306                 f"may be inflated by a stray box.",
307                 r.rally_number))
308
309     if stop_frame is not None and r.frame_max > stop_frame:
310         issues.append(Issue(
311             "high", "frame_out_of_range",
312             f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
313             f"last frame {stop_frame}.",
314             r.rally_number))
315
316     if i > 0:
317         prev = ordered[i - 1]
318         if r.start_time < prev.end_time:
319             issues.append(Issue(
320                 "blocker", "overlap",
321                 f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
322                 f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
323                 f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
324                 r.rally_number))
325             elif r.start_time - prev.end_time > max_gap_s:
326                 # A long unlabeled stretch between two rallies is a cheap, label-only
327                 # second opinion for the worst error (a missed rally). It won't catch a
328                 # rally MERGED into its neighbour's span (shot_density/inflated catches
329                 # that), but it does catch an omitted rally that left a visible hole.
330                 gap = r.start_time - prev.end_time
331                 issues.append(Issue(
332                     "medium", "possible_missed_rally",
333                     f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
334                     f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
335                     f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
336                     gap.",
337                     r.rally_number))
338
339     return issues
340
341 def convert(xml_path: str,
342            fps: Optional[float] = None,
343            video_path: Optional[str] = None,
344            step: Optional[int] = None,
345            allowed_reasons: Optional[set] = None) -> ConversionResult:

```

```

345     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
346
347     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
348     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
349     that silent guess is exactly the bug this adapter exists to undo.
350     """
351     meta, boxes = parse_cvat_boxes(xml_path)
352
353     if fps is None and video_path:
354         _, _, probed = probe_resolution_fps(video_path)
355         fps = probed
356     if fps is None or fps <= 0:
357         raise ValueError(
358             "Could not determine fps. Pass fps=... or a readable video_path so "
359             "frames can be converted to seconds (seconds = frame / fps).")
360
361     if step is None:
362         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
363
364     records = build_records(boxes, fps)
365     issues = detect_issues(records,
366                             stop_frame=meta.get("stop_frame"),
367                             allowed_reasons=allowed_reasons)
368     return ConversionResult(records=records, issues=issues, fps=fps, step=step,
369                             meta=meta)
370
371 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
372     """Write records as the canonical per-rally CSV that ``import-local`` reads."""
373     import csv
374     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
375     with open(out_path, "w", newline="", encoding="utf-8") as f:
376         w = csv.writer(f)
377         w.writerow(CANONICAL_COLUMNS)
378         for r in sorted(records, key=lambda x: x.rally_number):
379             w.writerow([
380                 r.rally_number,
381                 _mmss(r.start_time), _mmss(r.end_time),
382                 f"{r.start_time:.3f}", f"{r.end_time:.3f}",
383                 r.shots_count if r.shots_count is not None else "",
384                 r.ending_reason or "",
385                 r.last_shot_outcome or "",
386                 r.score_before or "", r.score_after or "", r.notes or "",
387             ])
388
389
390 def render_issue_report(result: ConversionResult, source: str) -> str:
391     """A human-readable issue report (for review / sending back to the vendor)."""
392     lines = [
393         f"CVAT rally conversion report ? {source}",
394         f"  fps used      : {result.fps:.3f}",
395         f"  sampling step  : {result.step} frames (~{result.step / result.fps:.2f}s)",
396         f"  rallies parsed : {len(result.records)}",
397         f"  issues        : {len(result.issues)} ({result.n_blockers} blocker)",
398         "",
399     ]
400     if not result.issues:
401         lines.append("No issues detected.")
402     else:
403         for sev in ("blocker", "high", "medium", "low"):
404             group = [i for i in result.issues if i.severity == sev]
405             if not group:
406                 continue
407             lines.append(f"[{sev.upper()}]")
408             for i in group:

```

```

409         lines.append(f" - ({i.code}) {i.message}")
410         lines.append("")
411     return "\n".join(lines)
412
413
414 # ----- #
415 # Validation reporting ? used by `curate validate-delivery`. #
416 # These turn a (records, issues) pair into the artifacts a human reviews: #
417 # a machine summary (JSON), a human report (md), and an annotatable sheet. #
418 # ----- #
419
420 def _parse_secs(value: Optional[str]) -> Optional[float]:
421     """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
422     s = (value or "").strip()
423     if not s:
424         return None
425     if ":" in s:
426         try:
427             total = 0.0
428             for part in s.split(":"):
429                 total = total * 60 + float(part)
430             return total
431         except ValueError:
432             return None
433     try:
434         return float(s)
435     except ValueError:
436         return None
437
438
439 def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
440     """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
441     just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
442     fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
443     CSV carries no frame info ? only the frame-derived checks become inapplicable.)
444     """
445     import csv as _csv
446     if not os.path.exists(csv_path):
447         raise FileNotFoundError(f"CSV not found: {csv_path}")
448     records: List[RallyRecord] = []
449     with open(csv_path, newline="", encoding="utf-8") as f:
450         reader = _csv.DictReader(f)
451         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
452 [])]
453         for raw in reader:
454             row = {(k or "").strip().lower(): v for k, v in raw.items()}
455             rn = safe_int(row.get("rally_number"), None)
456             if rn is None:
457                 continue
458             start = _parse_secs(row.get("start_time"))
459             if start is None:
460                 start = _parse_secs(row.get("start_mmss"))
461             end = _parse_secs(row.get("end_time"))
462             if end is None:
463                 end = _parse_secs(row.get("end_mmss"))
464             if start is None or end is None:
465                 continue
466             records.append(RallyRecord(
467                 rally_number=rn, start_time=start, end_time=end,
468                 shots_count=safe_int(row.get("shots_count"), None),
469                 ending_reason=(row.get("ending_reason") or "").strip() or None,
470                 last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
471                 score_before=(row.get("score_before") or "").strip() or None,
472                 score_after=(row.get("score_after") or "").strip() or None,
473                 notes=(row.get("notes") or "").strip() or None,

```



```

473         frame_min=-1, frame_max=-1, n_boxes=0,
474     ))
475     return records
476
477
478     def build_summary(records: List[RallyRecord], issues: List[Issue],
479                     source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
480         """Machine-readable validation summary (JSON / CI)."""
481         by_sev = {s: [i for i in issues if i.severity == s]
482                  for s in ("blocker", "high", "medium", "low")}
483         flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
484         return {
485             "source": source,
486             "fps": fps,
487             "n_rallies": len(records),
488             "n_issues": len(issues),
489             "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
490             "n_blockers": len(by_sev["blocker"]),
491             "passed": len(by_sev["blocker"]) == 0,
492             "flagged_rallies": flagged,
493             "issues": [{"severity": i.severity, "code": i.code,
494                        "rally_number": i.rally_number, "message": i.message} for i in
495             issues],
496         }
497
498     def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str)
499     -> str:
500         """Human validation report: scoreboard + issues by severity + rallies to review."""
501         n = len(records)
502         by_sev = {s: [i for i in issues if i.severity == s]
503                  for s in ("blocker", "high", "medium", "low")}
504         flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
505         clean = n - len(flagged)
506         lines = [
507             f"# Delivery validation ? {source}",
508             "",
509             f"- rallies: **{n}**",
510             f"- issues: **{len(issues)}** (blocker {len(by_sev['blocker'])}, "
511             f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low
512             {len(by_sev['low'])})",
513             f"- structurally-clean rallies: **{clean}/{n}**"
514             + (f" ({100 * clean // n}%) " if n else ""),
515             f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
516             "",
517         ]
518         if issues:
519             for sev in ("blocker", "high", "medium", "low"):
520                 grp = by_sev[sev]
521                 if not grp:
522                     continue
523                 lines.append(f"## {sev.upper()}")
524                 lines += [f"- ({i.code}) {i.message}" for i in grp]
525                 lines.append("")
526             else:
527                 lines.append("No issues detected. ?\n")
528         if flagged:
529             lines.append(f"## Rallies to review: {flagged}")
530         return "\n".join(lines)
531
532     def write_review_csv(records: List[RallyRecord], issues: List[Issue], out_path: str) ->
533     None:
534         """Auto-generate the annotatable manual-verification sheet (blank `true_` cols)."""
535         import csv

```

```

534         flags: Dict[int, List[str]] = {}
535     for i in issues:
536         if i.rally_number is not None:
537             flags.setdefault(i.rally_number, []).append(i.code)
538     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
539     cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
540            "ending_reason", "last_shot_outcome", "auto_flags",
541            "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
542            "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
543     with open(out_path, "w", newline="", encoding="utf-8") as f:
544         w = csv.writer(f)
545         w.writerow(cols)
546         for r in sorted(records, key=lambda x: x.rally_number):
547             w.writerow([
548                 r.rally_number, _mmss(r.start_time), _mmss(r.end_time),
549                 f"{r.duration:.1f}", r.shots_count if r.shots_count is not None else "",
550                 r.ending_reason or "", r.last_shot_outcome or "",
551                 ";".join(flags.get(r.rally_number, [])),
552                 "", "", "", "", "", "", "",
553             ])
554

```

4. user (2026-06-10T09:59:08.661Z)

No files found

5. user (2026-06-10T09:59:46.360Z)

```

tests\test_downloader.py
tests\test_local_import.py
tests\test_schemas.py
tests\test_validator.py
tests\test_cli.py
tests\test_config.py
tests\test_parsing.py
tests\test_db.py
tests\test_cvat_parser.py
tests\test_reporting.py
tests\test_reporting_faq.py
tests\test_normalize_annotations.py
tests\test_licensing_gate.py
tests\test_parser.py
tests\test_import_guard.py
tests\test_cvat_robustness.py

```

6. user (2026-06-10T09:59:51.586Z)

```

60:         <name>shot_count</name>
118:     <attribute name="shot_count">1</attribute>
130:     <attribute name="shot_count">1</attribute>
142:     <attribute name="shot_count">1</attribute>
192:     <attribute name="shot_count">7</attribute>
204:     <attribute name="shot_count">7</attribute>
216:     <attribute name="shot_count">7</attribute>
228:     <attribute name="shot_count">7</attribute>
240:     <attribute name="shot_count">7</attribute>
252:     <attribute name="shot_count">7</attribute>

```

[Showing results with pagination = limit: 10]

7. assistant (2026-06-10T09:59:56.654Z)

Confirmed ? the Roora XML uses singular `shot\_count`. Now let me look at the existing CVAT parser tests to match conventions.

8. user (2026-06-10T09:59:57.293Z)

```
1      """Tests for the CVAT rally adapter (src/parsers/cvat/rally_cvat.py).
2
3      The adapter's whole reason for existing is to (a) recompute rally time from the
4      box FRAME RANGE at the true fps ? ignoring the vendor's wrong start/end_time
5      attributes ? and (b) report content defects rather than silently fixing them.
6      These tests pin both behaviours, plus the end-to-end hand-off to import-local.
7      """
8      import csv as _csv
9
10     from src.parsers.cvat.rally_cvat import (
11         RallyRecord, parse_cvat_boxes, build_records, detect_issues, convert,
12         write_canonical_csv, load_canonical_csv, build_summary, render_validation_md,
13         write_review_csv,
14     )
15     from src.cli.curate import SportsDataCLI
16
17
18     # --- helpers -----
19
20     def _img_box(frame, attrs):
21         rows = "".join(f'<attribute name="{k}">{v}</attribute>' for k, v in attrs.items())
22         return (f'<image id="{frame}" name="frame_{frame:06d}" width="1920" height="1080">'
23             f'<box label="rally" xtl="1" ytl="1" xbr="2" ybr="2">{rows}</box></image>')
24
25
26     def _write_cvat(path, images, step=30, stop_frame=1000):
27         body = "".join(images)
28         xml = (
29             '<?xml version="1.0" encoding="utf-8"?><annotations><version>1.1</version>'
30             f'<meta><job><stop_frame>{stop_frame}</stop_frame>'
31             f'<frame_filter>step={step}</frame_filter></job></meta>'
32             f'{body}</annotations>'
33         )
34         path.write_text(xml, encoding="utf-8")
35         return str(path)
36
37
38     def _rec(rn, start, end, shots=5, reason="winner", fmin=None, fmax=None, n=3,
39 outcome=None):
40         return RallyRecord(
41             rally_number=rn, start_time=start, end_time=end, shots_count=shots,
42             ending_reason=reason, last_shot_outcome=outcome,
43             score_before=None, score_after=None, notes=None,
44             frame_min=fmin if fmin is not None else int(start * 60),
45             frame_max=fmax if fmax is not None else int(end * 60), n_boxes=n)
46
47     # --- the core fix: time from frames, vendor times ignored -----
48
49     def test_time_recomputed_from_frames_not_attributes(tmp_path):
50         # rally 1 spans source frames 60..120; the vendor's start/end attributes are
51         # deliberately wrong (the frame/900 bug). At 60 fps the truth is 1.0..2.0s.
52         xml = _write_cvat(tmp_path / "j.xml", [
53             _img_box(60, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4",
54                 "shots_count": "6.0", "ending_reason": "winner",
55                 "score_before (A-B)": "0-0", "notes": "clean"}),
56             _img_box(90, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
57             _img_box(120, {"rally_number": "1.0", "start_time": "0.2", "end_time": "1.4"}),
58         ])
59         res = convert(xml, fps=60.0)
60         assert len(res.records) == 1
61         r = res.records[0]
62         assert r.start_time == 1.0 and r.end_time == 2.0      # frames/fps, not 0.2/1.4
```

```

63         assert r.vendor_start_time == "0.2"                # preserved for transparency
64         assert r.shots_count == 6
65         assert r.ending_reason == "winner"
66         assert r.score_before == "0-0"                    # "(A-B)" alias resolved
67         assert r.notes == "clean"
68
69
70     def test_step_and_fps_read_from_meta_and_arg(tmp_path):
71         xml = _write_cvat(tmp_path / "j.xml", [
72             _img_box(0, {"rally_number": "1", "shots_count": "3"}),
73             _img_box(30, {"rally_number": "1"}),
74         ], step=30)
75         res = convert(xml, fps=60.0)
76         assert res.step == 30
77         assert res.fps == 60.0
78
79
80     def test_fps_required_when_unavailable(tmp_path):
81         xml = _write_cvat(tmp_path / "j.xml", [_img_box(0, {"rally_number": "1"})])
82         try:
83             convert(xml) # no fps, no video_path
84         except ValueError as e:
85             assert "fps" in str(e).lower()
86         else:
87             raise AssertionError("expected ValueError when fps cannot be determined")
88
89
90     # --- track-mode parsing + outside filtering -----
91
92     def test_track_mode_and_outside_filtering(tmp_path):
93         xml = (
94             '<?xml version="1.0"?><annotations><meta><job>'
95             '<stop_frame>1000</stop_frame><frame_filter>step=30</frame_filter></job></meta>'
96             '<track id="0" label="rally">'
97             '<box frame="60" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
98             '<attribute name="rally_number">1</attribute>'
99             '<attribute name="shots_count">4</attribute></box>'
100             '<box frame="120" outside="0" xtl="1" ytl="1" xbr="2" ybr="2">'
101             '<attribute name="rally_number">1</attribute></box>'
102             '<box frame="150" outside="1" xtl="1" ytl="1" xbr="2" ybr="2">' # terminator
103             '<attribute name="rally_number">1</attribute></box>'
104             '</track></annotations>'
105         )
106         p = tmp_path / "track.xml"
107         p.write_text(xml, encoding="utf-8")
108         meta, boxes = parse_cvat_boxes(str(p))
109         assert len(boxes) == 2                # the outside=1 terminator dropped
110         recs = build_records(boxes, fps=60.0)
111         assert recs[0].frame_max == 120        # not 150 (the terminator)
112         assert recs[0].end_time == 2.0
113
114
115     # --- issue detection: surfaced, not silently fixed -----
116
117     def test_overlap_is_a_blocker():
118         recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0)] # r2 starts before r1 ends
119         issues = detect_issues(recs)
120         overlaps = [i for i in issues if i.code == "overlap"]
121         assert len(overlaps) == 1 and overlaps[0].severity == "blocker"
122
123
124     def test_zero_duration_is_a_blocker():
125         recs = [_rec(1, 2.0, 2.0, fmin=120, fmax=120, n=1)]
126         issues = detect_issues(recs)
127         assert any(i.code == "zero_duration" and i.severity == "blocker" for i in issues)

```

```

128
129
130 def test_offvocab_ending_reason_flagged():
131     recs = [_rec(1, 1.0, 3.0, reason="into_net")]
132     issues = detect_issues(recs)
133     assert any(i.code == "ending_reason_vocab" for i in issues)
134     # the adopted forced/unforced terms are accepted, not flagged
135     ok = detect_issues([_rec(1, 1.0, 3.0, reason="forced_error")])
136     assert not any(i.code == "ending_reason_vocab" for i in ok)
137
138
139 def test_low_shot_density_flags_inflated_span():
140     recs = [_rec(1, 0.0, 16.0, shots=2)] # 2 shots over 16s -> 0.12/s
141     issues = detect_issues(recs)
142     assert any(i.code == "shot_density" for i in issues)
143
144
145 def test_outcome_vocab_flagged():
146     bad = detect_issues([_rec(1, 1.0, 3.0, outcome="long")])
147     assert any(i.code == "outcome_vocab" for i in bad)
148     # the agreed outcomes are accepted, not flagged
149     ok = detect_issues([_rec(1, 1.0, 3.0, outcome="out")])
150     assert not any(i.code == "outcome_vocab" for i in ok)
151
152
153 # --- end-to-end: emitted CSV imports via the existing import-local parser ---
154
155 def test_canonical_csv_round_trips_through_import_local(tmp_path):
156     xml = _write_cvat(tmp_path / "j.xml", [
157         _img_box(60, {"rally_number": "1", "start_time": "0.2", "end_time": "1.4",
158             "shots_count": "6", "ending_reason": "winner",
159             "last_shot_outcome": "in"}),
160         _img_box(120, {"rally_number": "1"}),
161         _img_box(180, {"rally_number": "2", "shots_count": "4",
162             "ending_reason": "forced_error", "last_shot_outcome": "out"}),
163         _img_box(240, {"rally_number": "2"}),
164     ])
165     res = convert(xml, fps=60.0)
166     assert res.n_blockers == 0
167     assert res.records[1].last_shot_outcome == "out" # parsed from the box attribute
168     out = tmp_path / "canonical.csv"
169     write_canonical_csv(res.records, str(out))
170
171     # Re-read with the actual import-local parser (self is unused -> None is safe).
172     items = SportsDataCLI._parse_local_annotations(None, "badminton", "vid1", str(out))
173     assert len(items) == 2
174     assert items[0].start_time == 1.0 and items[0].end_time == 2.0 # not 0.2/1.4
175     assert items[1].ending_reason == "forced_error"
176     assert items[1].last_shot_outcome == "out" # survives the round-trip into
the schema
177     assert items[1].start_time == 3.0 and items[1].end_time == 4.0
178
179
180 # --- validate-delivery: gap heuristic, CSV loading, report artifacts -----
181
182 def test_possible_missed_rally_gap_flagged():
183     # 20s of unlabeled time between two rallies -> flag (catches the 1:46-style miss)
184     recs = [_rec(1, 95.6, 98.0), _rec(2, 118.0, 119.6)]
185     issues = detect_issues(recs)
186     assert any(i.code == "possible_missed_rally" for i in issues)
187
188
189 def test_normal_gap_not_flagged():
190     recs = [_rec(1, 10.0, 14.0), _rec(2, 22.0, 26.0)] # 8s gap -> fine
191     issues = detect_issues(recs)

```

```

192         assert not any(i.code == "possible_missed_rally" for i in issues)
193
194
195     def test_load_canonical_csv_decimal_and_mmss(tmp_path):
196         p = tmp_path / "c.csv"
197         p.write_text(
198             "rally_number,start_time,end_time,start_mmss,end_mmss,shots_count,ending_reason,
199             last_shot_outcome\n"
200             "1,3.0,21.0,,,15,unforced_error,out\n"
201             "2,,,0:29.000,0:32.000,2,winner,in\n", encoding="utf-8")
202         recs = load_canonical_csv(str(p))
203         assert len(recs) == 2
204         assert recs[0].start_time == 3.0 and recs[0].last_shot_outcome == "out"
205         assert recs[1].start_time == 29.0 and recs[1].end_time == 32.0 # mm:ss fallback
206
207     def test_load_canonical_csv_keeps_rally_starting_at_zero(tmp_path):
208         # Regression: a rally that legitimately starts at 0.0s must not be dropped.
209         # The old `start_time or start_mmss` fallback treated the falsy 0.0 as "missing"
210         # and skipped the row when there was no start_mmss column to fall back to.
211         p = tmp_path / "c.csv"
212         p.write_text(
213             "rally_number,start_time,end_time,shots_count,ending_reason,last_shot_outcome\n"
214             "1,0.0,4.0,8,winner,in\n"
215             "2,10.0,14.0,6,forced_error,out\n"
216             "3,20.0,24.0,5,unforced_error,out\n", encoding="utf-8")
217         recs = load_canonical_csv(str(p))
218         assert len(recs) == 3 # the zero-start row 1 is not lost
219         assert recs[0].start_time == 0.0 and recs[0].end_time == 4.0
220
221
222     def test_validation_artifacts(tmp_path):
223         recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0, reason="into_net")] # overlap + bad
224         vocab
225         issues = detect_issues(recs)
226         summary = build_summary(recs, issues, source="vid", fps=59.94)
227         assert summary["n_rallies"] == 2
228         assert summary["passed"] is False and summary["n_blockers"] >= 1
229         assert 2 in summary["flagged_rallies"]
230         md = render_validation_md(recs, issues, "vid")
231         assert "BLOCKED" in md and "## BLOCKER" in md
232         out = tmp_path / "review.csv"
233         write_review_csv(recs, issues, str(out))
234         rows = list(_csv.DictReader(open(out, encoding="utf-8")))
235         assert "VERIFIED(Y/N)" in rows[0] and "auto_flags" in rows[0]
236         assert any("overlap" in r["auto_flags"] for r in rows)

```

9. user (2026-06-10T10:00:04.083Z)

No files found

10. user (2026-06-10T10:00:05.469Z)

```

646:     def convert_cvat(self, args):
1022:         "convert-cvat",
1080:         elif args.command == "convert-cvat":
1081:             cli.convert_cvat(args)

```

11. user (2026-06-10T10:00:11.412Z)

```

640         if skipped_noncomm:
641             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")

```

```

642         if skipped_empty:
643             print(f"Skipped (no annotations) : {skipped_empty}")
644         print(f"Manifest          : {manifest_path}\n")
645
646     def convert_cvat(self, args):
647         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
648
649         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
650         attributes) on the ingestion side: each rally's time is recomputed from
651         its frame range at the video's true fps, and content defects (overlaps,
652         zero-duration, off-vocabulary endings, coarse sampling) are written to an
653         issues report rather than silently "fixed". The emitted CSV imports via
654         `import-local`; any blocker issue must be resolved first, since the
655         importer's validator rejects overlapping / zero-duration rallies.
656         """
657         from src.parsers.cvat.rally_cvat import (
658             convert, write_canonical_csv, render_issue_report,
659         )
660         try:
661             result = convert(args.input, fps=args.fps,
662                             video_path=args.video_path, step=args.step)
663         except (FileNotFoundError, ValueError) as e:
664             print(f"Error: {e}")
665             sys.exit(1)
666
667         if not result.records:
668             print(f"Error: the CVAT export parsed to ZERO rallies ({args.input}). "
669                   f"Likely the wrong job was exported or the rally_number attribute was "
670                   f"renamed (boxes without a matched rally_number are skipped).")
671             sys.exit(1)
672
673         write_canonical_csv(result.records, args.out)
674
675         # Observability report next to the canonical CSV. Never raises.
676         from src.reporting import emit_delivery_report
677         src_name = os.path.splitext(os.path.basename(args.input))[0]
678         dur = (result.meta.get("stop_frame") / result.fps) if
679         (result.meta.get("stop_frame") and result.fps) else None
680         emit_delivery_report(
681             out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
682             records=result.records, issues=result.issues, fps=result.fps,
683             video_path=args.video_path,
684             video_duration_s=dur, source_format="cvat", step=result.step,
685         )
686         report = render_issue_report(result, os.path.basename(args.input))
687         if args.issues:
688             with open(args.issues, "w", encoding="utf-8") as f:
689                 f.write(report)
690         print(report)
691         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
692         if result.n_blockers:
693             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject "
694                   f"this file until they are resolved (by the vendor or manual "
695                   f"review).")
696         else:
697             print("\n No blockers. Next:")
698             print(f"    python -m src.cli.curate import-local --sport <sport> "
699                   f"    f"--video-path <video> --annotations-path {args.out} --fps "
700                   f"{result.fps:g}")
701
702     def validate_delivery(self, args):

```

12. user (2026-06-10T10:00:11.924Z)

```
1015     fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
1016                                help="Retries per video when a download fails or returns a
low format (default: 2)")
1017     fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
1018                                help="Base backoff between retries; grows linearly per
attempt (default: 20)")
1019
1020     # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
1021     cvat_parser = subparsers.add_parser(
1022         "convert-cvat",
1023         help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
1024     cvat_parser.add_argument("--input", required=True,
1025                              help="Path to the CVAT XML export (image- or track-mode)")
1026     cvat_parser.add_argument("--out", required=True,
1027                              help="Path to write the canonical per-rally CSV (import-
local input)")
1028     cvat_parser.add_argument("--video-path",
1029                              help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
1030     cvat_parser.add_argument("--fps", type=float,
1031                              help="Override fps instead of probing the video")
1032     cvat_parser.add_argument("--step", type=int,
1033                              help="Frame sampling step (default: read from CVAT meta,
else inferred)")
1034     cvat_parser.add_argument("--issues",
1035                              help="Path to also write the human-readable issues report")
1036
1037     # Validate-delivery command ? run the full check suite + emit a structured report
1038     vd_parser = subparsers.add_parser(
1039         "validate-delivery",
1040         help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
1041     vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
1042     vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
1043     vd_parser.add_argument("--video-path",
1044                              help="Source video; fps via ffprobe (when using --input)")
1045     vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
1046     vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
1047     vd_parser.add_argument("--out-dir", default="data/validation",
1048                              help="Where to write the report + review sheet")
1049
1050     # Export command
1051     export_parser = subparsers.add_parser(
1052         "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
1053     export_parser.add_argument("--out-dir", default="data/eval_export",
1054                                help="Output directory for per-video CSVs and
manifest.json")
```

13. assistant (2026-06-10T10:00:30.699Z)

I have the full picture. Let me check the other CVAT test file and any other consumers of the attrs dict before changing the normalization boundary.

14. user (2026-06-10T10:00:32.231Z)

```
tests\test_cvat_parser.py:11: RallyRecord, parse_cvat_boxes, build_records, detect_issues,
```



```

convert,
tests\test_cvat_parser.py:108: meta, boxes = parse_cvat_boxes(str(p))
tests\test_cvat_robustness.py:12: from src.parsers.cvat.rally_cvat import parse_cvat_boxes,
convert
tests\test_cvat_robustness.py:44: parse_cvat_boxes(str(p))
tests\test_cvat_robustness.py:52: meta, boxes = parse_cvat_boxes(str(zpath))
src\parsers\cvat\rally_cvat.py:134: def _attr_lookup(attrs: Dict[str, str], canonical: str) ->
Optional[str]:
src\parsers\cvat\rally_cvat.py:141: def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any],
List[Dict[str, Any]]]:
src\parsers\cvat\rally_cvat.py:227: rn = safe_int(_attr_lookup(b["attrs"],
"rally_number"), None)
src\parsers\cvat\rally_cvat.py:236: attrs = items[0]["attrs"]
src\parsers\cvat\rally_cvat.py:242: shots_count=safe_int(_attr_lookup(attrs,
"shots_count"), None),
src\parsers\cvat\rally_cvat.py:243: ending_reason=_attr_lookup(attrs,
"ending_reason"),
src\parsers\cvat\rally_cvat.py:244: last_shot_outcome=_attr_lookup(attrs,
"last_shot_outcome"),
src\parsers\cvat\rally_cvat.py:245: score_before=_attr_lookup(attrs, "score_before"),
src\parsers\cvat\rally_cvat.py:246: score_after=_attr_lookup(attrs, "score_after"),
src\parsers\cvat\rally_cvat.py:247: notes=_attr_lookup(attrs, "notes"),
src\parsers\cvat\rally_cvat.py:351: meta, boxes = parse_cvat_boxes(xml_path)

```

15. user (2026-06-10T10:00:32.653Z)

```

1      """validate-delivery / convert-cvat must FAIL (not silently PASS or traceback) on a CVAT
2      export that is empty, corrupt, or zipped.
3
4      Regression for: an empty CVAT (0 rallies) printed GATE: PASS; corrupt XML raised an
5      uncaught ParseError; the standard .zip delivery artifact was unsupported.
6      """
7      import argparse
8      import zipfile
9
10     import pytest
11
12     from src.parsers.cvat.rally_cvat import parse_cvat_boxes, convert
13     from src.cli.curate import SportsDataCLI
14
15
16     def _box(frame, attrs):
17         rows = "".join(f'<attribute name="{k}">{v}</attribute>' for k, v in attrs.items())
18         return (f'<image id="{frame}" name="f{frame:06d}" width="1920" height="1080">'
19                 f'<box label="rally" xtl="1" ytl="1" xbr="2" ybr="2">{rows}</box></image>')
20
21
22     def _xml(body):
23         return ('<?xml version="1.0" encoding="utf-8"?><annotations><version>1.1</version>'
24                 f'<meta><job><stop_frame>1000</stop_frame><frame_filter>step=30</frame_filter>'
25                 f'</job></meta>{body}</annotations>')
26
27
28     def _write(path, body):
29         path.write_text(_xml(body), encoding="utf-8")
30         return str(path)
31
32
33     def test_empty_cvat_converts_to_zero_records(tmp_path):
34         # Boxes whose rally_number attribute was renamed -> skipped -> zero rallies.
35         p = _write(tmp_path / "empty.xml", _box(30, {"wrong_name": "1"}))
36         result = convert(p, fps=30.0)
37         assert result.records == []
38

```

```

39
40 def test_corrupt_xml_raises_valueerror_not_parseerror(tmp_path):
41     p = tmp_path / "bad.xml"
42     p.write_text("<annotations><image><box>", encoding="utf-8") # truncated
43     with pytest.raises(ValueError):
44         parse_cvat_boxes(str(p))
45
46
47 def test_zip_delivery_is_parsed(tmp_path):
48     inner = _xml(_box(30, {"rally_number": "1"}) + _box(60, {"rally_number": "1"}))
49     zpath = tmp_path / "delivery.zip"
50     with zipfile.ZipFile(zpath, "w") as zf:
51         zf.writestr("annotations.xml", inner)
52     meta, boxes = parse_cvat_boxes(str(zpath))
53     assert len(boxes) == 2
54
55
56 def test_validate_delivery_blocks_on_empty_cvat(tmp_path):
57     p = _write(tmp_path / "empty.xml", _box(30, {"wrong_name": "1"}))
58     cli = SportsDataCLI()
59     args = argparse.Namespace(
60         input=p, csv=None, video_id="d", fps=30.0, video_path=None,
61         out_dir=str(tmp_path),
62         )
63     with pytest.raises(SystemExit) as exc:
64         cli.validate_delivery(args)
65     assert exc.value.code == 2 # GATE: BLOCKED, not PASS

```

16. user (2026-06-10T10:01:14.023Z)

Switched to a new branch 'fix/cvat-shot-count-alias'

17. assistant (2026-06-10T10:01:21.982Z)

Now the parser fix:

18. user (2026-06-10T10:01:25.603Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py has been updated successfully. (file state is current in your context ? no need to Read it back)

19. user (2026-06-10T10:01:41.673Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py has been updated successfully. (file state is current in your context ? no need to Read it back)

20. user (2026-06-10T10:01:56.251Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py has been updated successfully. (file state is current in your context ? no need to Read it back)

21. user (2026-06-10T10:02:04.730Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py has been updated successfully. (file state is current in your context ? no need to Read it back)

22. user (2026-06-10T10:02:09.556Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py has been updated successfully. (file state is current in your context ? no need to Read it back)

23. user (2026-06-10T10:02:22.192Z)

The file C:\Users\avidu\Projects\sports-data-collector\tests\test_cvat_parser.py has been updated successfully. (file state is current in your context ? no need to Read it back)

24. user (2026-06-10T10:02:31.552Z)

```
..... [ 52%]  
..... [100%]  
137 passed in 2.07s
```

25. assistant (2026-06-10T10:02:36.542Z)

All 137 tests pass, including the new regression test. Now re-run the converter on both Roora XMLs:

26. user (2026-06-10T10:02:39.577Z)

- (outcome_vocab) Rally 4: last_shot_outcome '0' not in ['in', 'n_a', 'net', 'out'].
- (outcome_vocab) Rally 5: last_shot_outcome '0' not in ['in', 'n_a', 'net', 'out'].
- (outcome_vocab) Rally 6: last_shot_outcome '0' not in ['in', 'n_a', 'net', 'out'].

Canonical CSV written: C:\Users\avidu\AppData\Local\Temp\roora\pickleball_rallies.csv (6 rallies)

No blockers. Next:

```
python -m src.cli.curate import-local --sport <sport> --video-path <video> --annotations-  
path C:\Users\avidu\AppData\Local\Temp\roora\pickleball_rallies.csv --fps 59.94
```

27. user (2026-06-10T10:02:41.112Z)

```
rallies parsed : 6  
issues        : 0 (0 blocker)
```

No issues detected.

Canonical CSV written: C:\Users\avidu\AppData\Local\Temp\roora\tabletennis_rallies.csv (6 rallies)

No blockers. Next:

```
python -m src.cli.curate import-local --sport <sport> --video-path <video> --annotations-  
path C:\Users\avidu\AppData\Local\Temp\roora\tabletennis_rallies.csv --fps 59.94
```

28. user (2026-06-10T10:02:53.114Z)

```
1 rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot  
_outcome,score_before,score_after,notes  
2 1,0:00.000,0:01.001,0.000,1.001,1,service_fault,0,,,  
3 2,0:11.011,0:18.519,11.011,18.519,7,unforced_error,0,,,  
4 3,0:31.532,0:34.535,31.532,34.535,2,unforced_error,0,,,  
5 4,0:55.556,0:58.058,55.556,58.058,2,unforced_error,0,,,  
6 5,1:08.068,1:15.075,68.068,75.075,5,winner,0,,,  
7 6,1:29.089,1:33.594,89.089,93.594,3,unforced_error,0,,,  
8
```

29. user (2026-06-10T10:02:53.199Z)

```
1 rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot  
_outcome,score_before,score_after,notes  
2 1,0:02.503,0:06.006,2.503,6.006,5,other,,,,out of table  
3 2,0:11.011,0:12.012,11.011,12.012,2,other,,,,net  
4 3,0:16.016,0:19.520,16.016,19.520,4,other,,,,  
5 4,0:28.529,0:33.033,28.529,33.033,6,other,,,,  
6 5,0:37.037,0:45.546,37.037,45.546,12,winner,,,,  
7 6,0:51.051,0:55.556,51.051,55.556,7,other,,,,  
8
```

30. user (2026-06-10T10:02:54.081Z)

```
95:          <name>last_shot_outcome</name>
122:      <attribute name="last_shot_outcome">0</attribute>
134:      <attribute name="last_shot_outcome">0</attribute>
146:      <attribute name="last_shot_outcome">0</attribute>
196:      <attribute name="last_shot_outcome">0</attribute>
208:      <attribute name="last_shot_outcome">0</attribute>
220:      <attribute name="last_shot_outcome">0</attribute>
232:      <attribute name="last_shot_outcome">0</attribute>
```

[Showing results with pagination = limit: 8]

31. user (2026-06-10T10:03:06.718Z)

```
M src/parsers/cvat/rally_cvat.py
M tests/test_cvat_parser.py
src/parsers/cvat/rally_cvat.py | 38 ++++++-----
tests/test_cvat_parser.py      | 19 ++++++
2 files changed, 51 insertions(+), 6 deletions(-)
```

32. user (2026-06-10T10:03:15.804Z)

```
[fix/cvat-shot-count-alias fcddcc7] fix(cvat): fold attribute names so shot_count (singular)
maps to shots_count
2 files changed, 51 insertions(+), 6 deletions(-)
```

33. user (2026-06-10T10:03:24.698Z)

```
remote:
remote: Create a pull request for 'fix/cvat-shot-count-alias' on GitHub by visiting:
remote:      https://github.com/avidullu/sports-data-collector/pull/new/fix/cvat-shot-count-alias
remote:
branch 'fix/cvat-shot-count-alias' set up to track 'origin/fix/cvat-shot-count-alias'.
To https://github.com/avidullu/sports-data-collector.git
 * [new branch]      fix/cvat-shot-count-alias -> fix/cvat-shot-count-alias
```

34. user (2026-06-10T10:03:41.691Z)

<https://github.com/avidullu/sports-data-collector/pull/18>

35. assistant (2026-06-10T10:03:56.953Z)

```
---
name: session-handoff
description: Living session handoff for sports-data-collector ? current state, next steps, ramp-
up kit
metadata:
  type: project
---
```

Session handoff ? sports-data-collector

Last updated: 2026-06-10

You are here

```
- [PR #18](https://github.com/avidullu/sports-data-collector/pull/18) **open** (branch
`fix/cvat-shot-count-alias`):
  CVAT parser silently dropped the Roora `shot_count` (singular) attribute because the alias
  table only knew `shots_count`. Fixed with tolerant attribute-name folding
  (`_fold_attr_name` in `src/parsers/cvat/rally_cvat.py`: case/separators/parenthesized
  hints/plural). Verified: 137 tests pass; re-converted both Roora XMLs
```

```
(`%TEMP%\roora\{pb,tt}\annotations.xml`) ? pickleball shots 1,7,2,2,5,3, tabletennis  
5,2,4,6,12,7.
```

Next steps / open threads

- Merge PR #18 when reviewed.
- Pickleball Roora delivery has `last\_shot\_outcome = "0"` on every rally (vendor data defect, surfaced as `outcome\_vocab` issues by the converter) ? may need to go back to the vendor.
- Pickleball dataset sourcing decision still pending ? see [[pickleball-dataset-options]].

Ramp-up kit

- `src/parsers/cvat/rally\_cvat.py` ? the CVAT?canonical-CSV adapter (module docstring explains why it exists).
- `tests/test\_cvat\_parser.py`, `tests/test\_cvat\_robustness.py` ? pinned behaviors.
- Run suite: `python -m pytest tests/ -q` (fast, ~2s).

Key decisions

- Vendor attribute-name drift is absorbed on OUR side via folding, not renegotiated with vendors (same philosophy as recomputing time from frames at true fps).
- Content defects (vocab, overlaps, gaps) are surfaced as Issues, never silently fixed.

36. user (2026-06-10T10:03:56.975Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-sports-data-collector\memory\session-handoff.md (file state is current in your context ? no need to Read it back)

37. assistant (2026-06-10T10:04:02.755Z)

Memory Index

- ? Start here: [Session handoff](session-handoff.md) ? current state, next steps, ramp-up kit
- [Pickleball dataset options](pickleball-dataset-options.md) ? no turnkey timed pickleball dataset exists; 3 options pending user decision (deferred 2026-05-26)

38. user (2026-06-10T10:04:02.757Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

39. user (2026-06-10T10:04:07.693Z)

<system-reminder>This memory is 15 days old. Memories are point-in-time observations, not live state ? claims about code behavior or file:line citations may be outdated. Verify against current code before asserting as fact.</system-reminder>

```
1      # Memory Index  
2  
3      - [Pickleball dataset options](pickleball-dataset-options.md) ? no turnkey timed  
pickleball dataset exists; 3 options pending user decision (deferred 2026-05-26)  
4
```

40. assistant (2026-06-10T10:04:12.887Z)

Memory Index

- ? Start here: [Session handoff](session-handoff.md) ? current state, next steps, ramp-up kit
- [Pickleball dataset options](pickleball-dataset-options.md) ? no turnkey timed pickleball dataset exists; 3 options pending user decision (deferred 2026-05-26)

41. user (2026-06-10T10:04:12.898Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-sports-data-collector\memory\MEMORY.md has been updated successfully. (file state is current in your context

? no need to Read it back)

42. assistant (2026-06-10T10:04:28.471Z)

Done ? the fix is up as [sports-data-collector#18](https://github.com/avidullu/sports-data-collector/pull/18) and verified against both Roora deliveries.

****Root cause:**** exactly the suspected mismatch. The Roora XMLs carry `

****Fix:**** rather than just adding one alias, I added a `\_fold\_attr\_name` normalizer applied to both incoming attribute names (at the parse boundary in `parse\_cvat\_boxes`) and the alias table: lowercase, unified space/hyphen/underscore separators, parenthesized hints dropped, and per-token plural tolerance. So `Shot Count`, `shot-count`, `shot\_count`, and `shots\_count` all match, and it generalizes the previously hardcoded `score\_before (a-b)` alias ? `score\_before(A-B)` without the space now works too. `shot\_count` is also listed explicitly to document the observed Roora naming.

****Verification:****

- New regression test `test\_attr\_names\_matched\_tolerantly` in [test_cvat_parser.py](tests/test_cvat_parser.py:70) with a minimal CVAT fixture using singular `shot\_count`, `Ending Reason`, and `score\_before(A-B)`.
- Full suite passes: 137 tests.
- Re-ran `convert-cvat --fps 59.94` on both XMLs: pickleball `shots\_count` is now 1,7,2,2,5,3 and tabletennis 5,2,4,6,12,7 ? matching the source annotations.

One thing worth flagging back to the vendor: the pickleball delivery has `last\_shot\_outcome` set to literal `""` on every rally (it's in the XML verbatim, unrelated to this bug). The converter correctly surfaces those as `outcome\_vocab` issues in the report rather than importing them silently ? but the field is effectively unusable for that video until the vendor re-annotates it. I've noted this in the session handoff alongside the PR state.

43. user (2026-06-10T10:05:54.664Z)

I just merged the PR!

44. assistant (2026-06-10T10:06:00.560Z)

Let me sync up local state and refresh the handoff to reflect the merge.

45. user (2026-06-10T10:06:07.157Z)

Your branch is up to date with 'origin/master'.
Switched to branch 'master'
From https://github.com/avidullu/sports-data-collector
e4dd8f1..db5e1c2 master -> origin/master
Updating e4dd8f1..db5e1c2
Fast-forward
src/parsers/cvat/rally_cvat.py | 38 ++++++-----
tests/test_cvat_parser.py | 19 ++++++
2 files changed, 51 insertions(+), 6 deletions(-)
warning: deleting branch 'fix/cvat-shot-count-alias' that has been merged to
'refs/remotes/origin/fix/cvat-shot-count-alias', but not yet merged to HEAD
Deleted branch fix/cvat-shot-count-alias (was fcddcc7).

46. user (2026-06-10T10:06:20.824Z)

(PowerShell completed with no output)

47. assistant (2026-06-10T10:06:25.974Z)

GitHub already deleted the remote branch on merge. Let me prune the stale tracking ref and

refresh the handoff:

48. user (2026-06-10T10:06:29.909Z)

Pruning origin

URL: <https://github.com/avidullu/sports-data-collector.git>

- * [pruned] origin/chore/reclassify-corpus-licenses
- * [pruned] origin/feat/manual-ingestion-normalizer
- * [pruned] origin/fix/cvat-empty-corrupt-zip-gate
- * [pruned] origin/fix/cvat-shot-count-alias
- * [pruned] origin/fix/import-local-golden-label-guard
- * [pruned] origin/fix/licensing-video-vs-annotation-gate

49. user (2026-06-10T10:06:33.824Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-sports-data-collector\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)

50. assistant (2026-06-10T10:06:47.135Z)

All cleaned up. Local master is synced with the merge (db5e1c2), the local and remote fix branches are deleted (plus a few stale tracking refs from earlier PRs got pruned), and the session handoff now records the PR as merged.

The one open thread from this work: the pickleball Roora delivery still has `last\_shot\_outcome = "0"` on all six rallies ? that's a vendor annotation defect the converter flags but can't fix, so it's worth raising with Roora if you need that field for the pickleball video.